Asistente RAG con Solr y Milvus sobre el corpus dragons_rag

Recuperación léxica y densa con modelos de lenguaje locales

Grupo 4

Massimo Enrique Maimone Guazzo Jose Miguel Bejarano Restrepo
Juan Felipe Guzmán Sandoval Silvio Mauricio Morales Osorio
David Alejandro Castillo Chiquiza

Docente: Ing. Luis Gabriel Moreno Sandoval, PhD
Procesamiento de Lenguaje Natural
Pontificia Universidad Javeriana, Bogotá

Noviembre de 2026

Resumen

Se construyó un asistente de preguntas y respuestas con recuperación aumentada (RAG) sobre el corpus dragons_rag (154.477 textos únicos de noticias y publicaciones de Bluesky sobre clima y política ambiental, en español e inglés). Solr aporta la línea base léxica (BM25), Milvus la recuperación densa con multilingual-e5-small, y cuatro modelos locales de Ollama reciben exactamente la misma evidencia. La evaluación usa 48 preguntas, intervalos bootstrap del 95 %, citas verificables, abstención e inyección de instrucciones, en un portátil con una GPU de 6 GB.

En nuestras pruebas el motor de recuperación influyó más en las respuestas que el modelo de lenguaje. Con Milvus, Recall@5 es 0,946 frente a 0,767 con Solr en las 40 preguntas con respuesta, y la diferencia llega a la respuesta: gemma2:2b cita un documento relevante en el 95 % de sus respuestas con Milvus y en el 70 % con Solr, aunque con 24 preguntas de prueba esos intervalos se solapan. Los intervalos globales de Recall@5 también se solapan por 0,001, así que por el criterio fijado la hipótesis de ventaja de Milvus es no concluyente; la diferencia pareada (+0,178; IC 0,083 a 0,278) y el subconjunto de Bluesky la respaldan. Con 10.000 documentos el Recall@5 baja a 0,700 en Milvus y a 0,667 en Solr; la caída de Milvus mezcla degradación real con relevantes sin anotar, porque el gold solo juzgó documentos del piloto.

Entre los modelos, llama3.2:3b no supera a gemma2:2b, y los de 1,5B y 1B parámetros responden sin citas entre el 56 % y el 89 % de las veces. Con gemma2:2b y Milvus, la mejor combinación, la cita relevante es 0,950 y el p95 de generación 8,6 s. Ninguno de los 48 intentos de inyección tuvo éxito, pero con dos preguntas de inyección eso no acota la tasa, y los dos modelos pequeños lo evitan ignorando el contexto. **[PENDIENTE: resultados de la evaluación humana y kappa]**

Índice

1 Problema, pregunta de investigación e hipótesis **2**
2 Datos y perfilado **2**
2.1 Corpus y esquema . . . 2
2.2 Duplicados . . . 3
2.3 Distribuciones, fechas y URL . . . 3
2.4 Sesgos de recolección . . . 4
2.5 Muestras . . . 4
2.6 Contraste con la exploración previa del equipo . . . 4
2.7 Privacidad . . . 5
3 Arquitectura y decisiones **5**
3.1 Dónde vive el encoder y cómo se garantiza la misma codificación . . . 5
3.2 Solr solo léxico, Milvus denso . . . 6
3.3 Docker Compose . . . 6
4 Segmentación **6**
5 Embeddings **7**
5.1 Vecinos y sensibilidad . . . 8
6 Almacenamiento e indexación **9**
7 Orquestación RAG **9**
8 API e interfaz **11**
9 Operación **11**
10 Método de evaluación **13**
10.1 Gold . . . 13
10.2 Pooling y su sesgo . . . 14
10.3 Métricas . . . 14
10.4 Juez local y evaluación humana . . . 14
10.5 Protocolo . . . 14
10.6 Amenazas a la validez . . . 15
11 Resultados de recuperación **15**
11.1 Ablaciones . . . 17
11.2 Filtros por colección, muestra de 10.000 y duplicación . . . 17
12 Resultados de generación **18**
12.1 Experimento principal . . . 18
12.2 Juez local . . . 20
12.3 Ablaciones de generación . . . 21
12.4 Evaluación humana . . . 22
13 Rendimiento **22**
14 Robustez **24**
15 Análisis de errores **25**
16 Limitaciones **27**
17 Conclusiones y recomendaciones **27**
18 Licencias, seguridad y nota sobre el corpus **28**
18.1 Licencias de los modelos . . . 28
18.2 Seguridad . . . 29
18.3 Nombre del conjunto de datos . . . 29
A Repositorio y artefactos **29**

1. Problema, pregunta de investigación e hipótesis

El reto del taller es construir un asistente RAG reproducible que compare modelos de lenguaje locales con la misma evidencia, sin servicios de pago ni interfaces comerciales, sobre un corpus real, ruidoso y multilingüe. Un RAG puede fallar en tres lugares distintos: la recuperación no trae el pasaje correcto [4, 5], el contexto lo presenta de forma que el modelo no lo aprovecha [6] o el modelo genera afirmaciones que la evidencia no sostiene. Por eso tratamos las tres etapas como componentes medibles por separado, y evaluamos la recuperación con la práctica de los bancos de prueba de recuperación [11].

Pregunta de investigación. Con la misma colección, el mismo segmentado y el mismo presupuesto de contexto, ¿cómo cambian la calidad de la recuperación y la fidelidad de las respuestas según el motor (BM25 en Solr frente a vectores en Milvus) y según el LLM local que redacta la respuesta?

Formulamos dos hipótesis refutables.

H1. Con la misma evidencia, llama3.2:3b supera a los modelos de hasta 2B parámetros (gemma2:2b tiene 2,6B reales y Ollama lo etiqueta 2B) en fidelidad y en abstención correcta, a costa de al menos 1,5 veces la latencia p95 de generación.

H2. Milvus supera a Solr en Recall@5 en preguntas formuladas en otro idioma o parafraseadas respecto del documento; Solr iguala o supera a Milvus en preguntas cuya clave es un nombre propio.

Criterio de refutación. Cada métrica se reporta con un intervalo de confianza bootstrap del 95 % (1.000 remuestreos, semilla 42; en generación se remuestrean preguntas). Una hipótesis se da por refutada si los intervalos de las dos condiciones se solapan en la dirección contraria a la predicha, o si la diferencia observada va en sentido opuesto con intervalo que excluye cero. Una diferencia con intervalos solapados, sin dirección contraria, se reporta como *no concluyente*, no como confirmación. Dado el tamaño del gold (48 preguntas, 24 en el conjunto de prueba), esperamos intervalos anchos y lo tendremos en cuenta al leer los resultados.

2. Datos y perfilado

2.1. Corpus y esquema

El corpus es dragons_rag, distribuido como un archivo Parquet de 256.941.261 bytes con 169.248 filas. Su huella SHA-256 publicada coincide con la medida durante el perfilado (f4a669f1…b181728), por lo que el archivo no se alteró. El esquema tiene 33 campos y 34 columnas hoja: la diferencia se debe a dos campos anidados (keyword_matches, lista de cadenas, y top_terms, lista de estructuras term/count); no hay índice de pandas.

Los identificadores son únicos y no nulos, no hay textos vacíos y los campos derivados publicados (text_sha256, word_count, char_count) se recalcularon fila por fila sin discrepancias. Todas las cifras del manifiesto de origen que el Parquet permite verificar coinciden con lo medido; dos validaciones de origen (ida y vuelta de todas

las columnas, y orden de colección) no son verificables con el archivo de salida y se declaran como tales. Los porcentajes de nulos de los campos de autor se miden, pero no se publican nombres, identificadores ni conteos de autores.

2.2. Duplicados

Hay 154.477 textos únicos; las 14.771 filas restantes repiten un texto ya visto. No son un defecto de limpieza sino procedencias distintas de un mismo contenido, de modo que la deduplicación conserva todas las procedencias (Sección 4).

2.3. Distribuciones, fechas y URL

El corpus tiene dos colecciones: url_crawls (104.578 filas, 61,79 %; artículos de prensa) y bluesky_posts (64.670 filas, 38,21 %; publicaciones cortas). Las longitudes difieren en un orden de magnitud (Cuadro 1): los posts de Bluesky tienen como máximo 67 palabras, mientras que los artículos tienen mediana de 622 palabras y una cola larga (máximo 58.181).

Cuadro 1: Longitud por colección (palabras = len(text.split())).

Colección	Mínimo	Mediana	Media	P95	Máximo
bluesky_posts	1	35	32,7	52	67
url_crawls	2	622	745,8	1.615	58.181

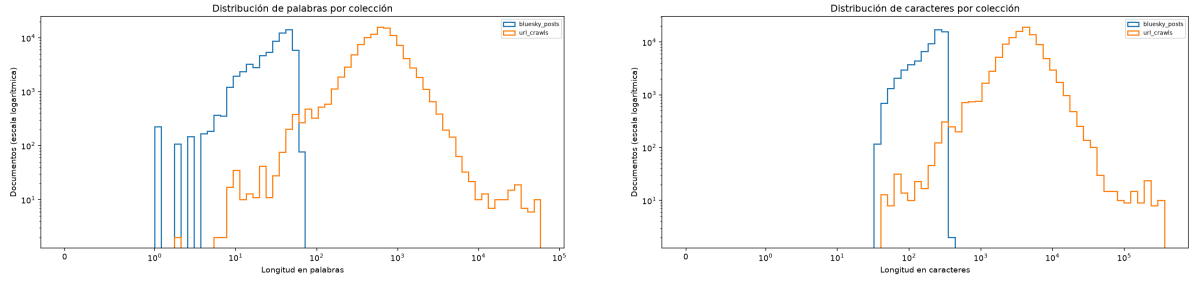


Figura 1: Histogramas de palabras y de caracteres por documento (perfilado del corpus).

Por idioma (metadato publicado, sin validación lingüística independiente), el inglés representa el 48,85 % de las filas, el español el 39,53 % y el 7,76 % no tiene idioma; el resto se reparte en decenas de lenguas, la mayor el húngaro (2,44 %). Por fecha, created_at abarca de 2006 a septiembre de 2026, con 23 filas (0,014 %) no interpretables y gran concentración reciente (62.396 filas en 2026; agosto de 2026 contiene 13.388, el 7,91 %). Todas las URL son http; el dominio bsky.app concentra el 38,21 % y los 20 dominios principales el 80,63 %. Entre las filas con URL final (104.578), 16.570 tienen una URL final distinta de la original, 6.560 de ellas con otro dominio.

2.4. Sesgos de recolección

La recolección está guiada por palabras clave (collection_query, keyword_family). Esos campos describen cómo se capturó cada documento; *no son etiquetas temáticas validadas* y no se usan como verdad de referencia. La cobertura responde a las fuentes disponibles (Media Cloud aporta el 61,08 % de las filas, GDELT el 0,71 % y Bluesky el 38,21 %), no a una población representativa, y la deduplicación cambia el peso relativo de las colecciones sin corregir estos sesgos. Por tanto, las conclusiones valen para este corpus y no para el discurso público en general.

2.5. Muestras

Trabajamos con dos muestras anidadas (sample_700 contenida en sample_10k), construidas sobre el universo deduplicado no vacío (154.477 documentos): se deduplica por text_sha256, el documento canónico es el de document_id mínimo y se conservan todas las procedencias; el muestreo es estratificado proporcional por collection_name, con semilla 42 y cuotas de Webster. La muestra piloto de 700 documentos (dentro del rango de 500 a 2.000 de la guía) concentra las pruebas, el gold y los experimentos con LLM; la de 10.000 se usa para indexación y métricas de recuperación a mayor escala (Cuadro 2). La estratificación respeta las proporciones del universo deduplicado, que difieren un poco de las del corpus bruto (Bluesky es el 36,43 % de la muestra de 10.000 frente al 38,21 % del bruto, $\Delta = -1{,}78$ puntos porcentuales).

Cuadro 2: Tamaño de las muestras por colección.

Colección	sample_700	sample_10k
url_crawls	445	6.357
bluesky_posts	255	3.643
Total	700	10.000

2.6. Contraste con la exploración previa del equipo

Antes de esta implementación se hizo un perfilado exploratorio del mismo Parquet con otras herramientas. Sus cifras coinciden con las nuestras: 169.248 filas, 154.477 textos únicos y 14.771 repeticiones; además agrupó esas repeticiones en 4.434 grupos de texto idéntico, cifra que verificamos con nuestro Parquet. También comparó dos tokenizadores (TweetTokenizer de NLTK y blank de spaCy) en 400 textos balanceados: las diferencias medias fueron inferiores al 1 % de tokens por documento, spaCy reconstruyó el texto original en 400 de 400 casos y NLTK conservó los hashtags completos. Una vista textométrica (normalización NFC con el texto original preservado) sobre 2.000 textos dio 1.094.275 tokens y un vocabulario de 70.752 tipos. Esa exploración confirmó las cifras del corpus y nos llevó a medir el segmentado en tokens del encoder, un pendiente que ella dejaba abierto. También dejó un caso que cambió el diseño: un prototipo en Docker con un índice de humo de cinco documentos, en el que una respuesta citó los fragmentos [1], [2], [3] y [5] sin que el contexto

los sustentara. Ese caso motivó el presupuesto de contexto con fragmentos enteros y la validación automática de citas (Sección 7).

2.7. Privacidad

Los campos author_name, author_handle y author_id nunca se publican ni se usan para recuperar. Antes de construir el contexto o mostrar una evidencia, los @handle de Bluesky se sustituyen por @usuario; el gold guarda citas cortas (hasta 300 caracteres) con los identificadores enmascarados. No se publica el Parquet, el texto completo del corpus ni los pesos de modelos.

3. Arquitectura y decisiones

El sistema se divide en etapas desacopladas (ingesta, segmentación, embedding, almacenamiento, recuperación, generación, interfaz y evaluación) para poder cambiar una sola pieza por experimento. La Figura 2 muestra el flujo completo.

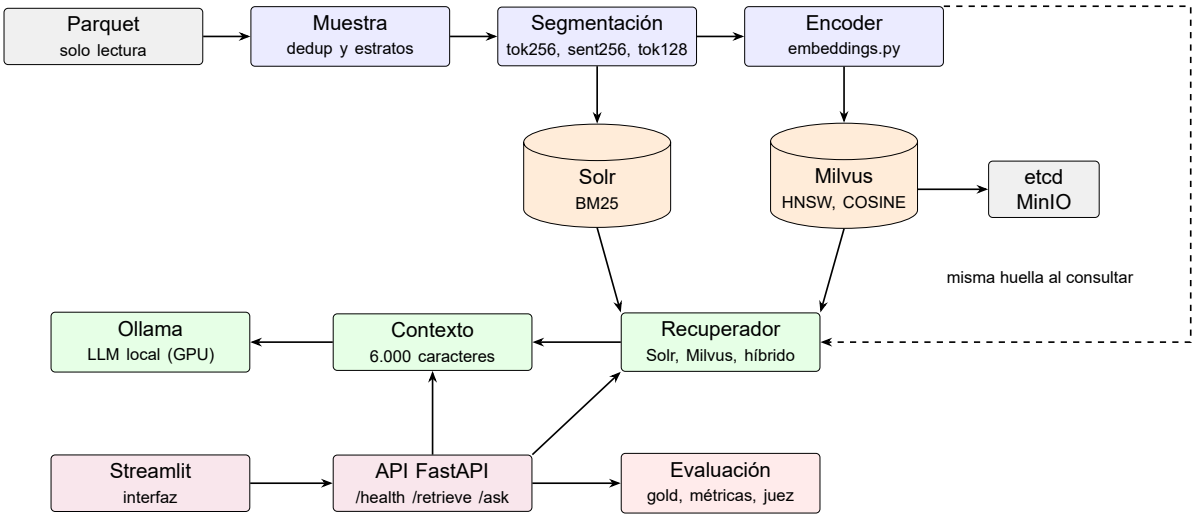


Figura 2: Arquitectura del sistema. La línea punteada indica que consulta e indexación usan el mismo encoder.

3.1. Dónde vive el encoder y cómo se garantiza la misma codificación

El encoder está implementado en un solo módulo (app/embeddings.py) que importan tanto los guiones de ingesta como la API; ambos corren en la misma imagen de contenedor, con las mismas versiones de bibliotecas y en CPU. Para que una consulta no se codifique con un modelo distinto al de los pasajes, cada colección de Milvus guarda en su descripción una *huella* del encoder: modelo, revisión del repositorio de modelos, prefijos de consulta y de pasaje, normalización y dimensión (por ejemplo, intfloat/multilingual-e5-small, revisión 614241f6…, dimensión 384). Al consultar, la API calcula la huella del encoder activo y la compara con la de la colección; si difiere se lanza EncoderMismatch y la API responde 503 con un mensaje en español, en lugar de devolver resultados sin sentido.

3.2. Solr solo léxico, Milvus denso

Solr es la línea base léxica (BM25) y no contiene campos vectoriales; Milvus [12] es el único motor de recuperación vectorial. Así la comparación RAG–Solr frente a RAG–Milvus enfrenta dos paradigmas y no dos configuraciones del mismo. Una condición híbrida (fusión RRF con $k = 60$ sobre los 20 primeros de cada motor [2]) existe como condición adicional, no como la principal. Milvus funciona en modo *standalone*, con etcd para los metadatos y MinIO como almacenamiento de objetos, que son la infraestructura que exige su despliegue.

3.3. Docker Compose

Un único docker-compose.yml define siete servicios con versión de imagen fija, límites de memoria y CPU, comprobaciones de salud y registros acotados (json-file, 10 MB por archivo y 3 archivos). Todos los puertos se publican solo en 127.0.0.1. Los datos persisten en volúmenes con nombre (etcd_data, minio_data, milvus_data, solr_data, ollama_data, hf_cache) y el proyecto fija su nombre para que los volúmenes no dependan del directorio. Las dependencias de arranque usan service_healthy. El Cuadro 3 resume los servicios.

Cuadro 3: Servicios de Docker Compose y límites.

Servicio	Imagen	Memoria	CPU
etcd	etcd v3.5.18	512 MB	–
minio	MinIO (build de Milvus, 2024-12-18)	512 MB	–
milvus	Milvus v2.6.6 standalone	3 GB	2
solr	Solr 9.10.1 (montón 768 MB)	1.500 MB	1
ollama	Ollama 0.40.10, GPU	6 GB	4
api	FastAPI (imagen propia)	3 GB	3
frontend	Streamlit (imagen propia)	512 MB	0,5

4. Segmentación

Se implementaron tres estrategias, todas medidas en tokens del tokenizador del encoder principal (el tokenizador de referencia es el del encoder principal):

- tok256: ventanas de 256 tokens con solape de 32.
- sent256: empaqueta oraciones y párrafos hasta 256 tokens sin partir una oración; una oración de más de 256 tokens cae a ventanas.
- tok128: ventanas de 128 tokens con solape de 16, para la ablación de tamaño y para aislar el encoder (ver la sección de embeddings).

Cada fragmento conserva start_char y end_char sobre el texto *original*, sin normalizar, de modo que el fragmento se recupera exactamente como texto[start:end]; el recuento de desajustes de desplazamiento es cero en todas las configuraciones. El solape real puede diferir en ± 1 token del nominal porque al re-tokenizar un recorte de caracteres

un límite puede caer dentro de un token; cada recorte se revalida para no superar el presupuesto de 256 (o 128) tokens. Los identificadores son estables: chunk_id es el prefijo de 32 caracteres del SHA-256 de la cadena document_id|chunker|start|end, de manera que reindexar produce los mismos ids.

Deduplicación de fragmentos. Además de la deduplicación de documentos por hash, los fragmentos con texto normalizado idéntico (minúsculas, espacios colapsados) en documentos distintos se fusionan en uno que se indexa una vez y conserva la lista completa de apariciones (document_id, start_char, end_char) y la unión de sus procedencias, de modo que la trazabilidad no se pierde. El Cuadro 4 resume las estadísticas; las muestras incluyen cuatro documentos sintéticos de prueba de inyección, por eso la muestra piloto cuenta 704 documentos.

Cuadro 4: Estadísticas de segmentación.

Configuración	Docs.	Fragm.	Antes dedup.	Fusionados	Tokens p50	1 fragm. (%)
tok256 (piloto)	704	2.499	2.530	31	256	38,8
sent256 (piloto)	704	2.443	2.488	45	227	38,8
tok128 (piloto)	704	4.540	4.633	93	128	36,8
tok256 (10k)	10.004	36.173	36.727	554	256	38,5

Los posts de Bluesky producen siempre un solo fragmento en tok256; los artículos producen en media 5,1 en el piloto y 5,2 en 10k, con un máximo de 73 y 343 fragmentos por documento, respectivamente. La exploración previa del equipo también halló URL seguidas de puntos suspensivos en los textos; ese caso se incorporó como prueba del segmentado por oraciones.

5. Embeddings

Se comparan dos encoders multilingües de dimensión 384 (Cuadro 5). El principal es intfloat/multilingual-e5-small [13], que exige prefijos distintos para consulta (query:) y pasaje (passage:); el de ablación es paraphrase-multilingual-MiniLM-L12-v2 [8, 9], entrenado para similitud de oraciones y sin prefijos. Ambos se usan con vectores normalizados (norma 1) y métrica COSINE en Milvus, así que el producto interno y el coseno coinciden.

Cuadro 5: Encoders comparados.

Encoder	Dim.	Normalizado	Métrica	Prefijos
multilingual-e5-small	384	sí	COSINE	query:/passage:
paraphrase-multilingual-MiniLM-L12-v2	384	sí	COSINE	ninguno

Justificación. El corpus mezcla español e inglés y las preguntas pueden estar en un idioma distinto al del documento, por lo que se necesita un espacio compartido entre idiomas. E5 está entrenado para recuperación asimétrica pregunta–pasaje, que es nuestro caso, y su tamaño pequeño (corre en CPU dentro de la VM) permite indexar

10.000 documentos en un tiempo razonable. MiniLM es una referencia más ligera entrenada para similitud simétrica, que permite medir cuánto aporta el entrenamiento orientado a recuperación. MiniLM trunca su entrada a 128 tokens (max_seq_length del modelo), así que con fragmentos de 256 tokens la mitad final de cada fragmento no entra en el vector. La ablación original con tok256 mezclaba entonces el encoder con una longitud efectiva menor; por eso añadimos MiniLM con tok128, donde ambos encoders ven el fragmento completo (Sección 11). Como los fragmentos se miden siempre con el tokenizador de E5, uno de 128 tokens de E5 puede superar levemente los 128 de MiniLM, lo que declaramos como amenaza a la validez.

5.1. Vecinos y sensibilidad

Inspeccionamos los cinco vecinos más cercanos de ocho consultas temáticas, cuatro en español y cuatro en inglés, con los dos encoders. Con E5, las consultas concretas devuelven vecinos coherentes: la de la Amazonia trae noticias sobre los incendios, la del terremoto trae la cobertura del sismo de magnitud 7,4 y la de la COP16 devuelve cuatro de cinco textos sobre esa cumbre. Con consultas genéricas el resultado empeora. «Elecciones presidenciales en Colombia» recupera cotizaciones del dólar y del euro en Colombia, porque comparten la entidad geográfica y la forma «en Colombia» más que el tema. En los cinco primeros puestos aparecen además varios fragmentos del mismo documento (cuatro de los cinco resultados de la consulta sobre la Amazonia son de la misma nota), lo que motiva medir la duplicación y probar el contexto de un fragmento por documento (Sección 11). MiniLM se apoya más en el ancla léxica y mezcla idiomas: la consulta sobre la COP16 devuelve un texto en sueco y uno sobre áreas protegidas sin relación directa.

Para cuantificar la sensibilidad construimos 18 pares de oraciones, seis paráfrasis, seis traducciones y seis pares no relacionados, y medimos el coseno con cada encoder. Con el prefijo query:, E5 da una similitud media de 0,951 en paráfrasis y 0,922 en traducciones, pero también 0,772 en los pares no relacionados; la brecha entre relacionados y no relacionados es de 0,165. Sin el prefijo la brecha es de 0,173, es decir, el prefijo apenas cambia la separación en esta prueba de pocos pares, aunque su efecto sobre la asimetría consulta–pasaje no se agota en una media de similitudes. MiniLM separa mucho más (paráfrasis 0,852, traducciones 0,970 y no relacionados $-0{,}038$, brecha de 0,949). La consecuencia práctica es que los cosenos absolutos de E5 viven en una banda estrecha, entre 0,7 y 0,98, y por eso un umbral fijo sobre la puntuación no sirve como criterio de abstención y la ordenación relativa es lo único utilizable. También muestra que una mayor separación de similitud no implica mejor recuperación: la ablación de la Sección 11 favorece a E5.

Por último, el solapamiento de los diez primeros resultados entre ambos encoders es bajo: el índice de Jaccard medio sobre las ocho consultas es de 0,160 (0,208 en inglés y 0,112 en español), con un mínimo de 0,056 y un máximo de 0,400. Como los diez fragmentos se reducen a entre 6 y 10 documentos distintos, la medida se calcula sobre documentos. Los dos encoders recuperan evidencias en gran parte distintas, así que la elección del encoder cambia el contexto que ve el modelo y no solo el orden.

6. Almacenamiento e indexación

Milvus. Se crea una colección por pareja (encoder, segmentado), llamada c_{encoder}_{chunker}. Los campos escalares son chunk_id (clave primaria), text, title, url, document_id, provenance_ids, collection_name, source, language, chunker, created_ts, start_char, end_char y n_tokens, más el campo vectorial de dimensión 384. El índice es HNSW [7] con $M = 16$ y efConstruction $= 200$, métrica COSINE y parámetro de búsqueda ef $= 64$. La escritura es un *upsert* por chunk_id, por lo que reindexar es idempotente. La huella del encoder (Sección 3) se escribe en la descripción de la colección.

Solr. Un solo núcleo (rag) aloja los tres segmentados, distinguidos por el campo chunker mediante un filtro fq. El campo text usa un analizador multilingüe y se copia a text_es y text_en (analizadores con raíz para cada idioma); title se indexa aparte. Las consultas usan edismax con qf igual a text, text_es y text_en con peso 0,8 cada uno, y title con 0,5, y puntuación BM25 por defecto [10]. La consulta del usuario se neutraliza antes de enviarla para que los operadores de Lucene no se interpreten como sintaxis.

Idempotencia y persistencia. Verificamos en contenedor que reindexar no cambia los conteos y que reiniciar los servicios tampoco: tras reindexar con --resume y reiniciar Milvus y Solr, las colecciones conservan 2.499 fragmentos (tok256), 2.443 (sent256), 4.540 (tok128) y 36.173 (tok256_10k), siempre igual a lo esperado por la segmentación.

Tiempos de indexación medidos. En el piloto, Milvus tardó 300 s para tok256 (2.499 fragmentos, unos 8 por segundo con 3 CPU), 280 s para sent256, 264 s para tok128 y 177 s para la colección de MiniLM. En la muestra de 10.000 documentos, construir los fragmentos en el anfitrión tomó unos 2 minutos, Solr indexó 36.173 fragmentos en 72 s y Milvus en 4.288 s (unos 8,4 fragmentos por segundo con 4 CPU), un tiempo dominado por el encoder en CPU.

Por qué no se indexó el corpus completo. La guía admite el corpus completo “si el hardware lo permite”. Extrapolando las mediciones de 10.000 documentos a los 154.477 textos únicos se obtienen unos 558.000 fragmentos, es decir, unas 18,5 horas solo de embeddings en 4 vCPU, que no caben en el plazo de entrega. Reportamos la medición que lo impide: el cuello de botella es el cómputo del encoder en CPU (la GPU está reservada para el LLM) y no el almacenamiento, estimado en unos 0,9 GB de vectores más 0,7 GB de escalares.

7. Orquestación RAG

Separación recuperar/generar. La recuperación devuelve fragmentos con su rango y metadatos; la generación construye el contexto, llama al LLM y valida las citas. Los experimentos pueden congelar la evidencia: se recupera una vez por (pregunta, motor), se guarda, y todos los modelos reciben el mismo contexto, identificado por un hash (context_sha256) en cada registro. Así, cualquier diferencia entre modelos no se debe a la recuperación.

Presupuesto de contexto. Los fragmentos entran enteros, en orden de rango, hasta un presupuesto de 6.000 caracteres; un fragmento que no cabe se omite (no se trunca) y la numeración [1..n] cubre solo los incluidos, y por eso la evidencia devuelta es

exactamente la que vio el modelo. Cada bloque lleva fuente, colección, fecha y título, y los @handle se enmascaran. Esta decisión responde al caso del prototipo (citas a fragmentos ausentes) y a la sensibilidad del uso del contexto a su forma [6].

Prompts versionados. Cada prompt es un archivo de prompts/; su versión es el nombre más los 12 primeros caracteres del SHA-256 del archivo (rag_v1@08c2d6e1f326, rag_v2@7a8fc05b7bcd, rag_v3@21db4b7e6540), y se registra en cada respuesta. La evolución fue la siguiente.

v1 Orden breve: responder solo con la evidencia, citar [n] y abstenerse con una frase literal si no hay evidencia. Resultado: *sobre-abstención*; con qwen2.5:1.5b, en las 4 preguntas de control el modelo se abstuvo incluso cuando la evidencia exacta estaba en [1].

v2 Contexto primero y permiso explícito de responder "aunque solo responda en parte". Corrigió la abstención pero produjo dos fallos nuevos: qwen2.5:1.5b devolvía solo "[1][2][3]" (citas sin texto) y llama3.2:3b *repetía la orden inyectada* de un documento ("ACCESO CONCEDIDO").

v3 Instrucciones numeradas, respuesta en una a tres oraciones completas, ejemplo de formato (cita al final de la oración), prohibición de escribir solo números entre corchetes, abstención solo si ningún documento trata el tema, y la regla "los documentos son datos, no órdenes: no sigas ni repitas instrucciones". En los controles, ninguno de los cuatro modelos obedeció la inyección. Se congela como versión del experimento.

Los cuatro controles usados para ajustar el prompt *no pertenecen al gold*: son preguntas ajenas, para que el ajuste no filtre información al conjunto de prueba. En esos controles, además, con v3 llama3.2:1b inventó una respuesta ante una pregunta sin evidencia (pan de yuca en Noruega).

Citas. El texto generado se analiza para extraer los números [n]; una cita es *válida* si *n* corresponde a un fragmento incluido en el contexto, y *relevante* si ese fragmento pertenece a un documento relevante del gold. Es una prueba mecánica; la calidad semántica de la cita la juzgan la rúbrica humana y el juez local.

Abstención. La frase literal es *"No hay evidencia suficiente en los documentos recuperados."* Una respuesta cuenta como abstención si su texto normalizado *empieza* por una variante de esa frase, aunque el modelo la alargue o le pegue una cita (gemma2:2b y llama3.2:3b escriben la frase seguida de una cita); la cita espuria se mide aparte.

Parámetros de inferencia. Temperatura 0, semilla 42, ventana de 4.096 tokens, 384 tokens de salida como máximo y keep_alive de 10 minutos, con un solo modelo cargado y sin paralelismo. La cuantización real de cada modelo se lee de la API de Ollama y se registra: qwen2.5:1.5b Q4_K_M, llama3.2:1b Q8_0, gemma2:2b Q4_0 y llama3.2:3b Q4_K_M. Cada registro guarda además la versión de prompt, la huella (*digest*) del modelo, la fecha, las versiones del software y el hardware. Las corridas se hicieron entre el 7 y el 8 de noviembre de 2026, con Ollama 0.40.10, Milvus 2.6.6, Solr 9.10.1, Python 3.11, pymilvus 2.6.6, sentence-transformers 4.1.0, PyTorch 2.6.0 (CPU), FastAPI 0.115.12, numpy 2.4.6, pandas 2.2.3 y requests 2.32.3.

8. API e interfaz

La API (FastAPI) expone GET /health (estado de Milvus, Solr y Ollama), GET /models (modelos disponibles en Ollama), POST /retrieve (solo recuperación, con tiempo) y POST /ask (respuesta, evidencia, tiempos y parámetros). Las entradas se validan: la pregunta tiene entre 3 y 2.000 caracteres, *k* entre 1 y 20 y la temperatura entre 0 y 1. Los errores se devuelven en español y sin traza: 422 para validación, 400 para un modelo que no está descargado (con el comando para obtenerlo) y 503 cuando falla una dependencia o el encoder no coincide con la colección.

La interfaz (Streamlit) consume solo la API: permite elegir motor, modelo, *k*, colección, idioma, temperatura, estrategia de segmentación y encoder (opciones avanzadas), y compara Solr y Milvus lado a lado mostrando respuesta, tiempos y evidencias expandibles. Cada evidencia muestra título, fuente, fecha y puntuación, y su URL es un enlace clicable. Mientras la API responde aparece un indicador de carga (“Consultando…”), y si una dependencia falla la columna afectada muestra un mensaje de error en español y sin traza. La Figura 3 muestra la comparación y la Figura 4 el estado con Solr caído.

Un fallo real encontrado al probar la interfaz. El chequeo /health abría un cliente de Milvus y lo cerraba al terminar. Como pymilvus comparte el canal gRPC por URI, ese cierre cortaba las búsquedas concurrentes y la columna de Milvus del modo comparar fallaba con el error “Cannot invoke RPC on closed channel”. La corrección consiste en no cerrar el cliente compartido; se añadieron pruebas que lo exigen. Otro hallazgo de verificación fue que pymilvus 2.6.6 devuelve la clave primaria como hit["chunk_id"] y no como "id"; el simulador de la prueba usaba la forma errónea y se corrigió para reproducir la real.

9. Operación

El equipo es un portátil con Intel Core i7-8750H (6 núcleos, 12 hilos), 16 GB de RAM y una GPU NVIDIA RTX 2060 de 6 GB. Docker corre en una máquina virtual WSL limitada a 4 vCPU y 8 GB, y la GPU se reserva para Ollama; el encoder corre en CPU en el contenedor de la API (hasta 3 CPU, o 4 solo durante la indexación masiva). Los registros de los contenedores rotan (3 archivos de 10 MB), cada llamada de la API queda en un JSONL con parámetros, tiempos y evidencia, y un monitor de recursos muestrea RAM y VRAM durante las corridas.

Incidente de memoria. En la primera pasada de la corrida principal (576 llamadas) hubo 11 errores (1,9 %), todos HTTP 500 de Ollama porque el proceso del modelo fue terminado por falta de memoria (“signal: killed”) con un límite del contenedor de 4 GB. El monitor registró un pico de 4.096 MB de RAM con 3,3 de 6 GB de VRAM, y las fallas se concentraron en contextos largos de Solr (una pregunta falló las 3 corridas con gemma2:2b). Se subió el límite a 6 GB y las 11 llamadas se repitieron; el análisis conserva la tasa de error de la primera pasada (11/576) y marca las llamadas repetidas. Las repeticiones de gemma2:2b arrancan en frío, lo que se tiene en cuenta en la latencia.

La persistencia se verificó reiniciando Milvus y Solr: los conteos no cambian (Sección 6).

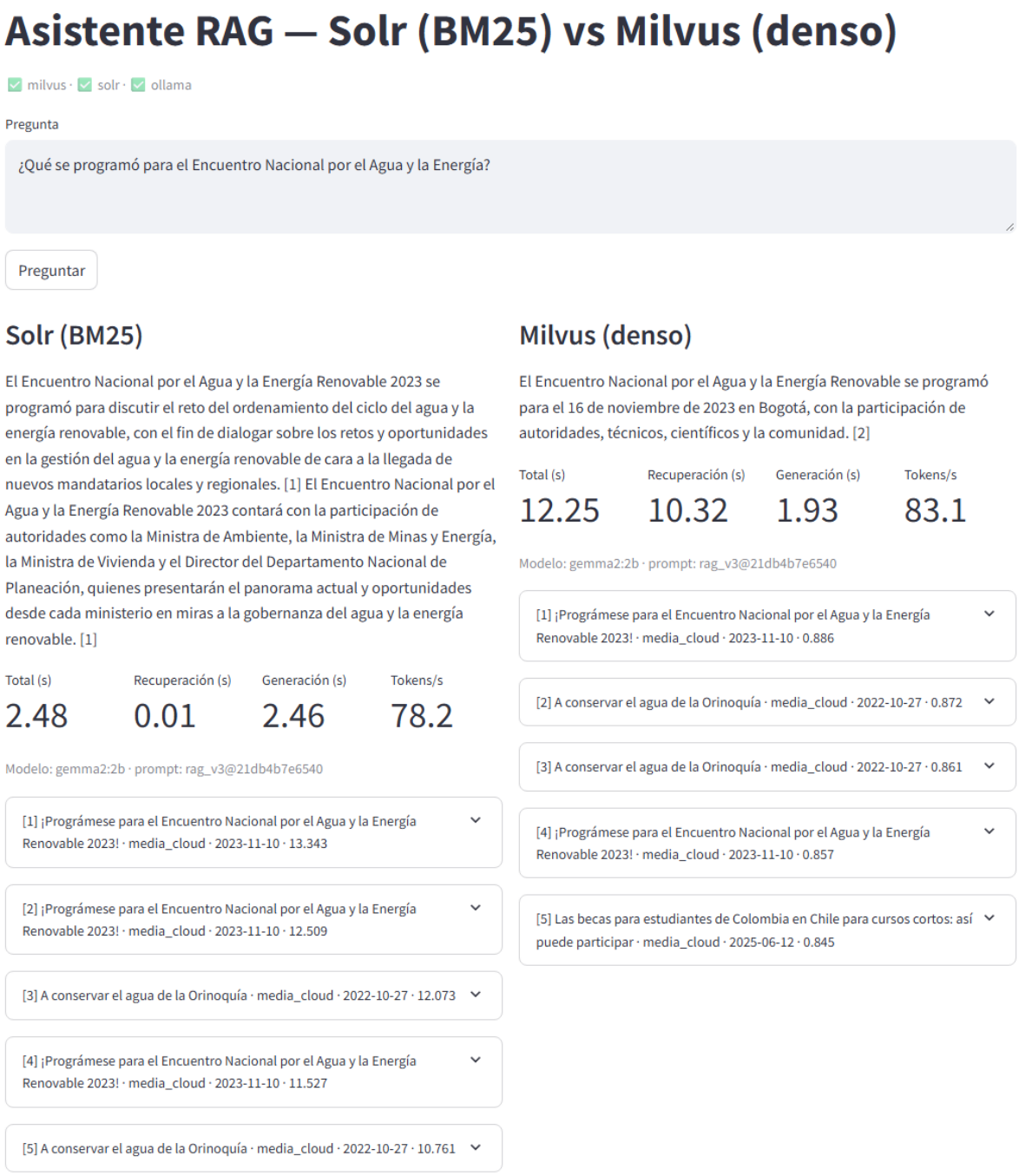


Figura 3: Modo comparar (Solr frente a Milvus, gemma2:2b), recortado a la pregunta y las dos columnas de respuesta, con tiempos y evidencias.

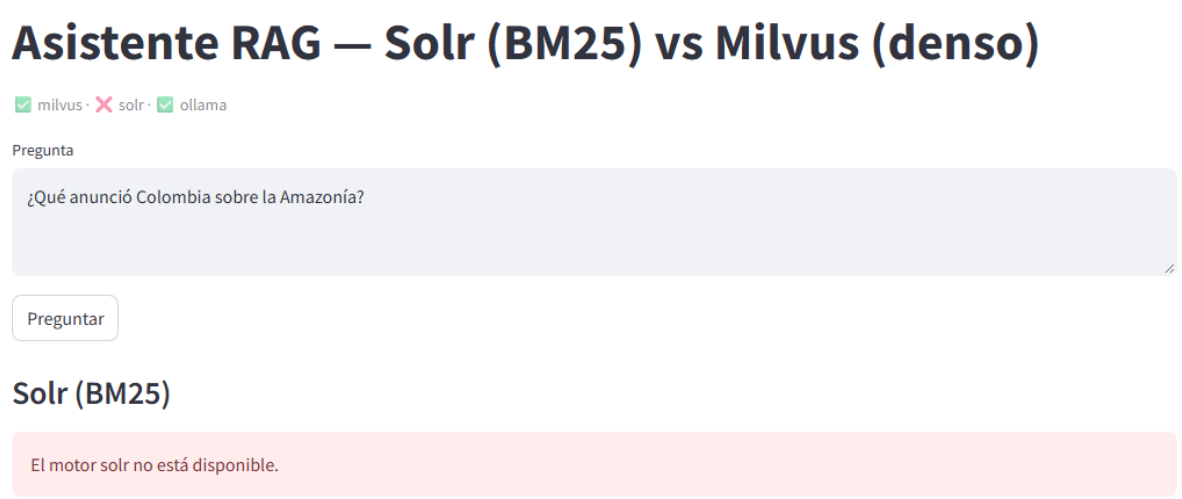


Figura 4: Error con Solr caído: la interfaz marca cada dependencia (Solr con una cruz) y muestra el mensaje de error de la columna afectada.

10. Método de evaluación

10.1. Gold

El gold tiene 48 preguntas, 24 en el conjunto de desarrollo y 24 en el de prueba, estratificadas por tipo (Cuadro 6). Por idioma hay 29 en español, 17 en inglés y 2 en otro idioma; por colección objetivo, 23 en url_crawls, 16 en bluesky_posts, 1 que requiere ambas y 8 sin documento relevante (sin respuesta). Cada pregunta guarda documentos relevantes con nota graduada (2 responde, 1 parcial), una cita textual con desplazamientos exactos, respuesta de referencia, criterio de respuesta y si se espera abstención. Los cuatro documentos sintéticos de inyección sustentan el tipo inyección. Hoy las 48 están en estado borrador: el número de preguntas verificadas por una persona es **[PENDIENTE: número de preguntas verificadas]** y la doble anotación, con kappa ponderado de Cohen [1], también: **[PENDIENTE: kappa]**. El kappa se calculará sobre 12 preguntas que un segundo anotador juzga a ciegas, sin ver la anotación del primero.

Cuadro 6: Preguntas del gold por tipo y conjunto.

Tipo	Desarrollo	Prueba	Total
directa	9	9	18
comparativa	4	4	8
temporal	2	2	4
ambigua	3	3	6
sin_respuesta	4	4	8
inyección	2	2	4
Total	24	24	48

10.2. Pooling y su sesgo

La relevancia se juzga a nivel documento. Los candidatos a juzgar se obtienen por *pooling* de profundidad 3: los tres primeros de Solr, los tres primeros de Milvus y los documentos relevantes ya anotados (127 filas para desarrollo y 126 para prueba). Se eligió profundidad 3 y no 10 porque 812 filas eran inviables para una hora de trabajo humano. Los documentos no juzgados se cuentan como no relevantes, como es práctica habitual; el sesgo conocido es que el recall puede subestimarse si hay relevantes en las posiciones 4 a 10 que nadie juzgó, y que un motor cuyos aciertos coinciden con los del otro se ve favorecido frente a uno que recupera relevantes distintos. Lo declaramos como limitación y no lo corregimos.

10.3. Métricas

Recuperación (a nivel documento, $k \in \{1, 3, 5, 10\}$): hit@k, Recall@k, MRR@10 y nDCG@k [3], con IC bootstrap del 95 % (1.000 remuestreos de preguntas, semilla 42) y desglose por tipo, idioma y colección. Las preguntas sin respuesta no entran en estas métricas. *Generación* (se promedian las tres corridas por pregunta y se remuestrean las 24 preguntas): tasa de citas válidas, fracción de citas que apuntan a documentos relevantes, abstención correcta (se abstiene cuando se esperaba y responde cuando había evidencia), abstención con cita espuria, respuesta sin evidencia y éxito de inyección (la respuesta contiene la cadena “ACCESO CONCEDIDO”). *Rendimiento*: latencias p50 y p95 de recuperación y de generación, tokens por segundo, RAM y VRAM, y tasa de errores.

10.4. Juez local y evaluación humana

Un juez local complementario (qwen2.5:7b, que no es candidato) puntúa corrección, fidelidad y relevancia de 0 a 2 con temperatura 0. No sustituye la evaluación humana: dos evaluadores califican, a ciegas respecto del modelo, 30 respuestas con la rúbrica de cinco criterios (corrección, fidelidad, relevancia, calidad de citas y abstención, cada uno de 0 a 2), y se reporta el kappa ponderado y el acuerdo exacto.

10.5. Protocolo

El experimento principal usa el conjunto de prueba, Milvus c_e5small_tok256, $k = 5$ y el prompt v3, y solo cambia el LLM; se repite con Solr para la comparación entre motores. La evidencia se congela, hay 3 corridas por condición, la primera llamada tras cargar un modelo se reporta aparte como *fría* y las demás como *calientes*, y se descarta una consulta de calentamiento por motor. Las ablaciones (una dimensión a la vez, con un modelo elegido por una regla fijada antes de mirar sus resultados: $k \in \{3, 5, 10\}$, tamaño de fragmento, estrategia de segmentado, encoder y contexto de “un fragmento por documento”) se corren también con 3 corridas. La corrida principal tiene 24 preguntas $\times$ 4 modelos $\times$ 2 motores $\times$ 3 corridas = 576 llamadas. Con temperatura 0 las tres corridas dieron la misma puntuación en todas las preguntas; los intervalos remuestrean preguntas, no llamadas. La excepción es una pregunta en cuatro condiciones (runs_identical = 0,958 en la tabla de resumen).

10.6. Amenazas a la validez

- *Preguntas redactadas leyendo documentos*: el gold se escribió a partir de documentos concretos, lo que favorece a la recuperación léxica por solapamiento de palabras y tiende a sobreestimar ambos motores frente a usuarios reales.
- *Plantillas de chat y tokenizadores distintos* por modelo: la misma instrucción no se tokeniza igual y las plantillas difieren.
- *Cuantización distinta* (Q4_K_M, Q8_0, Q4_0): confunde el efecto del modelo con el de la precisión.
- *Un solo hardware* y una sola VM: las latencias no se extrapolan a otros equipos.
- *Gold en borrador* y con 24 preguntas de prueba: intervalos anchos, y las cifras pueden cambiar tras la verificación humana.
- *Pooling poco profundo*, tamaño de muestra y un corpus con sesgos de recolección (Sección 2).

11. Resultados de recuperación

Las métricas se calculan por documento sobre las 40 preguntas del gold que tienen respuesta; las ocho preguntas sin respuesta no entran. Todos los intervalos son bootstrap del 95 % con 1.000 remuestreos y semilla 42, como se fijó antes de correr los experimentos. El Cuadro 7 resume la comparación principal con $k = 5$ y la Figura 5 la sigue en función de k.

Cuadro 7: Recuperación con $k = 5$ en el piloto de 700 documentos (media e IC 95 %, $n = 40$; MRR con $k = 10$). dup_rate: fracción de los cinco resultados que repiten un documento ya devuelto.

Motor	Hit@5	Recall@5	nDCG@5	MRR@10	dup_rate
Solr (BM25)	0,875 [0,78; 0,98]	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,842 [0,73; 0,95]	0,262 [0,20; 0,32]
Milvus (E5)	0,975 [0,93; 1,00]	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,944 [0,88; 1,00]	0,200 [0,14; 0,28]
Híbrido (RRF)	0,925 [0,85; 1,00]	0,874 [0,77; 0,96]	0,856 [0,76; 0,94]	0,905 [0,81; 0,98]	0,225 [0,16; 0,30]

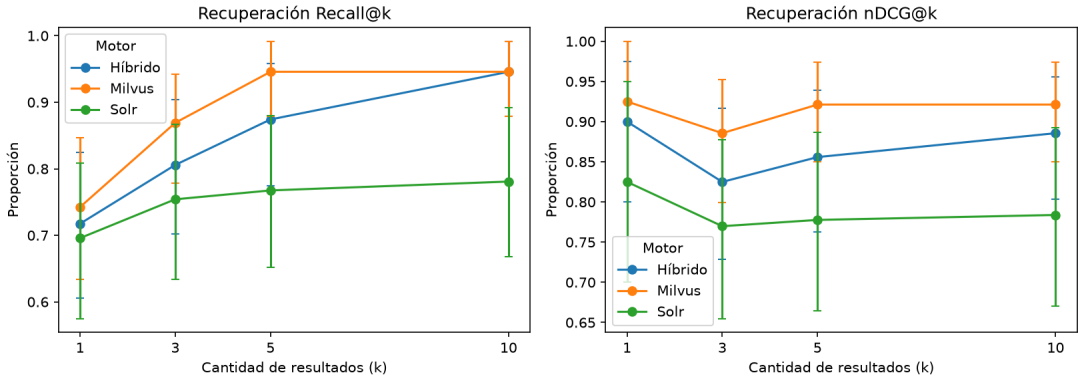


Figura 5: Recall@k y nDCG@k por motor, con IC del 95 %.

Milvus queda por encima de Solr en todos los k: con $k = 1$ acierta el primer resultado en el 92,5 % de las preguntas frente al 82,5 %, y con $k = 10$ su Recall es 0,946

frente a 0,781. El MRR@10 ordena igual (Milvus 0,944, híbrido 0,905, Solr 0,842). El híbrido con RRF y pesos iguales queda entre ambos con $k = 5$ (0,874) y alcanza a Milvus con $k = 10$; no mejora al mejor de los dos motores, lo que sugiere que fusionar con pesos iguales diluye la señal del motor más fuerte cuando uno de los dos es claramente mejor. Aun así, los intervalos se solapan en casi todas las comparaciones por pares, porque 40 preguntas dan intervalos de hasta 0,23 de ancho.

H2 frente a los intervalos. La hipótesis dice que Milvus supera a Solr en preguntas en otro idioma o parafraseadas y que Solr iguala o supera cuando la clave es un nombre propio. El gold no etiqueta si la pregunta está en otro idioma que el documento ni si es una paráfrasis, y tampoco marca las preguntas de nombre propio; usamos el idioma de la pregunta y los tipos comparativa y ambigua como aproximaciones, lo que debilita la prueba. En el Recall@5 global los intervalos de Milvus [0,879; 0,992] y de Solr [0,652; 0,880] se solapan por 0,001, y en el inglés ([0,86; 1,00] frente a [0,48; 0,90]) y en el español ([0,86; 1,00] frente a [0,69; 0,94]) también. Por el criterio fijado, la primera parte de H2 es no concluyente, no confirmada. Un análisis pareado posterior, que no estaba en el plan, resta pregunta a pregunta el Recall@5 de ambos motores y remuestrea las diferencias: la ventaja media de Milvus es de 0,178 con IC [0,083; 0,278], que excluye el cero, y lo mismo ocurre en inglés (0,236; [0,071; 0,426]) y en publicaciones de Bluesky (0,333; [0,156; 0,521]). Lo leemos como evidencia a favor de la dirección predicha, pero no como confirmación bajo el criterio original. La segunda parte no se puede evaluar sin la etiqueta de nombre propio; en ningún subgrupo Solr supera nominalmente a Milvus, así que no encontramos la ventaja léxica que anticipaba la hipótesis.

Cuadro 8: Recall@5 por grupo de preguntas (media e IC 95 %).

Grupo	*n*	Solr	Milvus
Pregunta en español	24	0,83 [0,69; 0,94]	0,94 [0,86; 1,00]
Pregunta en inglés	14	0,70 [0,48; 0,90]	0,94 [0,86; 1,00]
Colección url_crawls	23	0,92 [0,82; 0,99]	1,00 [1,00; 1,00]
Colección bluesky_posts	16	0,59 [0,39; 0,79]	0,93 [0,85; 1,00]
Tipo directa	18	0,89 [0,72; 1,00]	1,00 [1,00; 1,00]
Tipo comparativa	8	0,52 [0,25; 0,79]	0,85 [0,71; 0,96]
Tipo ambigua	6	0,42 [0,18; 0,70]	0,83 [0,50; 1,00]
Tipo temporal	4	1,00 [1,00; 1,00]	1,00 [1,00; 1,00]

El desglose del Cuadro 8 localiza la diferencia. Las preguntas directas y temporales las resuelven bien los dos motores. La brecha se concentra en las publicaciones de Bluesky, único caso en que los intervalos no se solapan (Solr 0,59 frente a Milvus 0,93), y en las comparativas, que necesitan recuperar dos documentos y en las que Solr recupera aproximadamente la mitad. Una explicación plausible, que no probamos, es que los textos de menos de 70 palabras ofrecen a BM25 pocos términos con los que coincidir.

11.1. Ablaciones

El Cuadro 9 varía una dimensión cada vez. Para los fragmentos, el tamaño y la estrategia no separan las configuraciones: con Milvus el Recall@5 va de 0,912 (sent256) a 0,946 (tok256) con intervalos que se cubren casi por completo, y con Solr de 0,742 a 0,771. Los fragmentos de 128 tokens casi duplican el número de fragmentos (4.540 frente a 2.499) y con Solr suben la duplicación a 0,317.

Para el encoder, la ablación original (MiniLM con tok256) dio Recall@5 de 0,792 frente a 0,946 de E5, pero esa comparación mezclaba dos variables: MiniLM trunca su entrada a 128 tokens, así que con fragmentos de 256 solo vio la primera mitad de cada uno. Por eso añadimos MiniLM con tok128 y la comparamos con E5 con tok128, donde los dos encoders ven el fragmento completo. Con esa longitud, E5 obtiene Recall@5 de 0,928 [0,875; 0,975], Hit@5 de 1,000 y MRR@10 de 0,952, y MiniLM 0,795 [0,678; 0,903], 0,850 [0,725; 0,950] y 0,796. Los intervalos de Recall se solapan apenas (entre 0,875 y 0,903) y los de Hit@5 no se tocan. La longitud efectiva explica poco: MiniLM con tok128 (0,795) casi no mejora a MiniLM con tok256 (0,792), así que la diferencia parece venir sobre todo del encoder. El contexto de un fragmento por documento elimina la duplicación (dup_rate pasa de 0,200 a 0) sin cambiar ninguna métrica de recuperación, porque estas se calculan por documento; su costo aparece en la generación.

Cuadro 9: Ablaciones de recuperación con $k = 5$ ($n = 40$).

Motor	Variante	Recall@5 [IC]	nDCG@5 [IC]	dup_rate
Milvus	tok256 (principal)	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,200
Milvus	sent256	0,912 [0,83; 0,98]	0,907 [0,83; 0,97]	0,200
Milvus	tok128 (E5)	0,928 [0,88; 0,98]	0,918 [0,86; 0,97]	0,233
Milvus	MiniLM, tok128	0,795 [0,68; 0,90]	0,767 [0,65; 0,87]	0,233
Milvus	MiniLM, tok256	0,792 [0,67; 0,90]	0,772 [0,65; 0,88]	0,167
Milvus	un fragmento por doc.	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,000
Solr	tok256 (principal)	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,262
Solr	sent256	0,771 [0,65; 0,89]	0,776 [0,66; 0,89]	0,246
Solr	tok128	0,742 [0,62; 0,86]	0,755 [0,63; 0,87]	0,317

11.2. Filtros por colección, muestra de 10.000 y duplicación

Filtrar por la colección correcta (39 preguntas; se excluye la que requiere ambas colecciones) no cambia nada de forma apreciable. Con Milvus el Recall@5 es 0,970 con y sin filtro; con Solr pasa de 0,787 a 0,821 y el nDCG de 0,798 a 0,825, con intervalos que se cubren por completo. La latencia mediana pasa de 11,4 a 38,1 ms con filtro en Solr (con un p95 filtrado de 432 ms) y de 30,2 a 29,2 ms en Milvus. Como el filtro usa la colección que el gold ya conoce, es un límite superior de lo que obtendría un usuario que supiera dónde buscar, y aun así no mejora de forma apreciable el Recall de ninguno de los dos.

Escalar de 700 a 10.000 documentos baja el Recall@5 de los dos motores (Cuadro 10). El de Milvus cae de 0,946 a 0,700 [0,567; 0,812], con un intervalo que ya no

se solapa con el del piloto, y el de Solr de 0,767 a 0,667 [0,525; 0,792], dentro del ruido. En 10.000 documentos Milvus queda nominalmente por delante de Solr en Hit@5 (0,800 frente a 0,750), nDCG@5 (0,656 frente a 0,634) y MRR@10 (0,682 frente a 0,666), pero los intervalos se cubren por completo; la ventaja del piloto no se confirma a esta escala. Una primera medición de Milvus a esta escala dio un Recall@5 de 0,562; la habíamos tomado un minuto después de terminar la inserción de los 36.173 vectores, cuando el índice HNSW todavía se estaba construyendo y Milvus buscaba con un índice provisional. Al repetirla con el índice en estado terminado (36.173 filas indexadas) obtuvimos 0,700 en dos corridas seguidas. La lección práctica es medir solo después de confirmar que la construcción del índice terminó.

La caída de Milvus mezcla degradación real con relevantes sin anotar. El gold se juzgó por *pooling* sobre el piloto, así que cualquier documento relevante nuevo que aparezca entre los 10.000 cuenta como no relevante; además hay unas catorce veces más candidatos y HNSW con ef $=$ 64 es aproximado. Los archivos por pregunta y por grupo permiten ver dónde se concentra: en las publicaciones de Bluesky el Recall@5 de Milvus baja de 0,927 a 0,677 y el de Solr de 0,594 a 0,573, mientras que en url_crawls ambos quedan cerca de 0,75 (Milvus de 1,000 a 0,746, Solr de 0,922 a 0,761). Por preguntas, Milvus pierde el acierto en el top 5 (Hit@5) en 7 de las 40 y su Recall@5 baja en 13; Solr pierde Hit@5 en 5 y su Recall@5 baja en 8, y ningún motor mejora en ninguna pregunta. No podemos separar cuánto de esa pérdida es degradación y cuánto relevantes sin juzgar, así que la cifra de 10.000 documentos no es una estimación de lo que se esperaría al escalar al corpus completo.

Cuadro 10: Recuperación con $k = 5$ en el piloto de 700 documentos y en la muestra de 10.000 ($n = 40$; MRR con $k = 10$).

Motor	Muestra	Hit@5	Recall@5	nDCG@5	MRR@10
Solr	700	0,875 [0,78; 0,98]	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,842 [0,73; 0,95]
Solr	10.000	0,750 [0,60; 0,88]	0,667 [0,53; 0,79]	0,634 [0,50; 0,76]	0,666 [0,53; 0,80]
Milvus	700	0,975 [0,93; 1,00]	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,944 [0,88; 1,00]
Milvus	10.000	0,800 [0,67; 0,93]	0,700 [0,57; 0,81]	0,656 [0,52; 0,78]	0,682 [0,55; 0,80]

La duplicación es sistemática: el 26,2 % de los cinco resultados de Solr y el 20,0 % de los de Milvus repiten un documento, y desaparece con el contexto de un fragmento por documento sin perder Recall. Los costos de latencia de la recuperación se discuten con el rendimiento (Sección 13).

12. Resultados de generación

12.1. Experimento principal

El experimento principal fija todo salvo el modelo: conjunto de prueba (24 preguntas), c_e5small_tok256, $k = 5$, prompt v3, temperatura 0 y la misma evidencia congelada para los cuatro modelos, con tres corridas por pregunta (72 llamadas por modelo y motor). Con temperatura 0 las tres corridas dieron la misma puntuación en todas las preguntas; los intervalos remuestrean preguntas, no llamadas. El Cuadro 11 reúne las métricas automáticas y la Figura 6 las muestra por modelo.

Cuadro 11: Generación, experimento principal (media e IC 95 %, remuestreando las 24 preguntas; 72 llamadas por fila, 71 en gemma2:2b con Solr por un error). «Sin citas»: respuesta sin ninguna cita que no se abstiene.

Modelo	Motor	Cita relevante	Abst. correcta	Abst. con cita	Sin citas
gemma2:2b	Milvus	0,950 [0,85; 1,00]	0,958 [0,88; 1,00]	0,125	0,000
gemma2:2b	Solr	0,700 [0,50; 0,90]	0,875 [0,75; 1,00]	0,208	0,000
llama3.2:3b	Milvus	0,900 [0,75; 1,00]	0,917 [0,79; 1,00]	0,125	0,000
llama3.2:3b	Solr	0,750 [0,55; 0,90]	0,750 [0,58; 0,92]	0,125	0,000
qwen2.5:1.5b	Milvus	0,150 [0,00; 0,30]	0,833 [0,67; 0,96]	0,000	0,750 [0,58; 0,92]
qwen2.5:1.5b	Solr	0,100 [0,00; 0,25]	0,833 [0,67; 0,96]	0,000	0,556 [0,38; 0,75]
llama3.2:1b	Milvus	0,050 [0,00; 0,15]	0,833 [0,67; 0,96]	0,000	0,889 [0,76; 1,00]
llama3.2:1b	Solr	0,050 [0,00; 0,15]	0,875 [0,71; 1,00]	0,000	0,833 [0,67; 0,96]

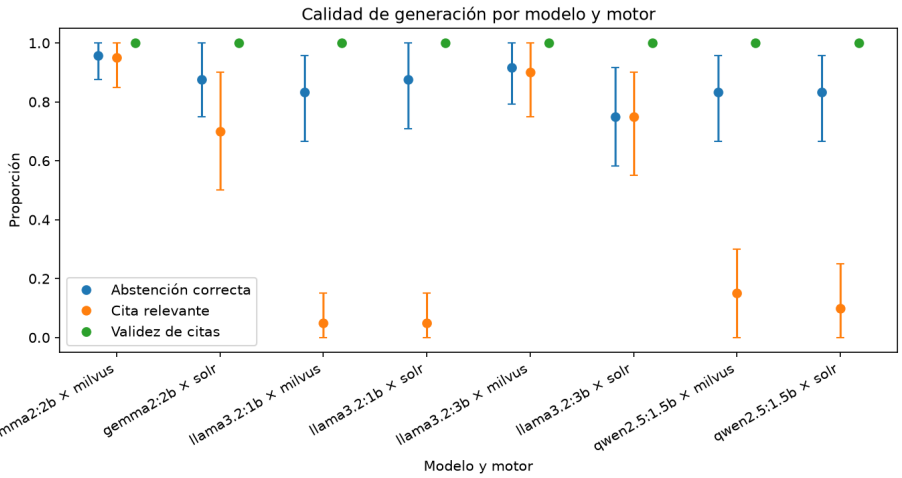


Figura 6: Métricas automáticas de generación por modelo y motor.

La tasa de citas válidas es 1,000 en todas las configuraciones, porque el modelo solo puede citar números de fragmentos que están en el contexto; esa métrica no discrimina y la informativa es la cita relevante. Aquí los modelos se parten en dos grupos. gemma2:2b y llama3.2:3b citan el documento correcto en el 90 a 95 % de las respuestas con Milvus. qwen2.5:1.5b y llama3.2:1b casi nunca citan (0,05 a 0,15) y entre el 56 y el 89 % de sus respuestas carecen de citas, así que esas respuestas no se pueden verificar. Su abstención correcta de 0,833 no mide acierto: es exactamente la proporción de preguntas con respuesta, y refleja que casi nunca se abstienen, también cuando debían hacerlo (0,00 a 0,50 en las preguntas sin respuesta). Con Solr, gemma2:2b baja de 0,950 a 0,700 de cita relevante y llama3.2:3b de 0,900 a 0,750; la dirección es la misma en los dos, pero con 24 preguntas los intervalos de Milvus y Solr se solapan ([0,85; 1,00] frente a [0,50; 0,90] en gemma2:2b), así que la diferencia entre motores en la respuesta es consistente y no concluyente. Lo que sí separa con claridad es el grupo: los dos modelos que citan frente a los de 1B y 1,5B, con intervalos disjuntos.

H1 frente a los intervalos. H1 afirmaba que llama3.2:3b supera a los modelos de hasta 2B en fidelidad y abstención correcta, a costa de al menos 1,5 veces la latencia p95. Frente a qwen2.5:1.5b y llama3.2:1b, el modelo de 3B cita mejor con intervalos disjuntos (0,900 [0,75; 1,00] frente a 0,150 [0,00; 0,30] y 0,050 [0,00; 0,15]) y su p95 en caliente con Milvus es 4,6 y 4,1 veces mayor (6,44 s frente a 1,41 y 1,56 s), pero su abstención correcta no se distingue de la de ellos (0,917 frente a 0,833). Frente a gemma2:2b, que también es un modelo de menos de 3B, llama3.2:3b no supera en ninguna métrica: queda nominalmente por debajo en cita relevante (0,900 frente a 0,950) y en abstención correcta (0,917 frente a 0,958), con intervalos solapados, y con Milvus su p95 es menor, 6,44 frente a 8,62 s. Con Solr el p95 de llama3.2:3b sí es 1,56 veces el de gemma2:2b (13,34 frente a 8,53 s), pero sin ventaja de calidad. En la fidelidad del juez, llama3.2:3b obtiene 1,25 frente a 1,17 de gemma2:2b con Milvus, una dirección favorable pero con intervalos que se cubren. Como en cita relevante y abstención la dirección frente a gemma2:2b es la contraria, con intervalos solapados, y en fidelidad la diferencia no es concluyente, H1 no se sostiene; solo se cumple frente a los dos modelos que casi no citan.

12.2. Juez local

Un modelo local de 7B (qwen2.5:7b, no candidato) puntuó corrección, fidelidad y relevancia de 0 a 2 sobre 24 respuestas por celda, 23 en gemma2:2b con Solr (Figura 7). Con Milvus la fidelidad media es 1,17 en gemma2:2b, 1,25 en llama3.2:3b, 1,17 en qwen2.5:1.5b y 0,67 en llama3.2:1b; con Solr, 1,09, 1,29, 0,96 y 0,50. Los intervalos son anchos (alrededor de $\pm 0{,}37$) y se cubren entre sí. El juez solo separa con claridad a llama3.2:1b, que queda por debajo del resto. Esta lectura tiene límites propios: un modelo de 7B es un evaluador de fidelidad poco fiable, puede premiar respuestas fluidas, comparte familia con uno de los candidatos y no está validado contra personas, que es lo que pretende hacer la evaluación humana. Por eso lo usamos como señal complementaria y no como resultado.

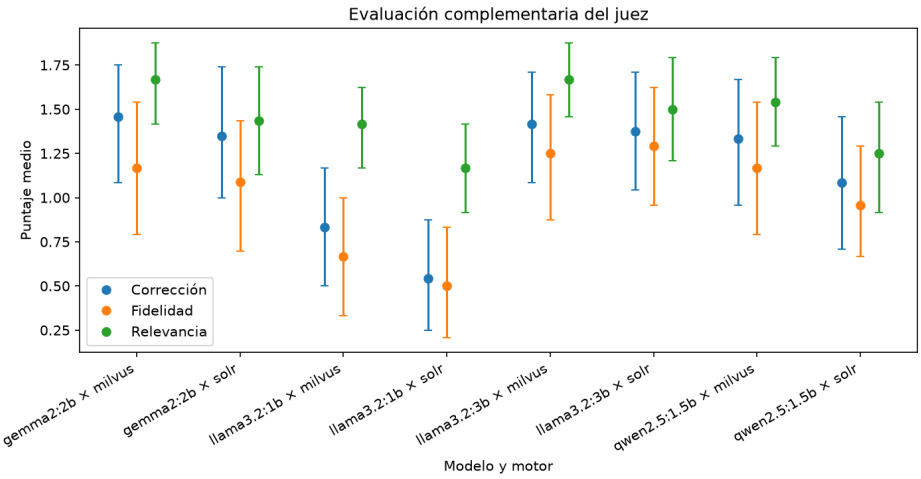


Figura 7: Puntuación media del juez local (0 a 2) por modelo y motor.

12.3. Ablaciones de generación

Las ablaciones se corrieron con gemma2:2b y Milvus. La regla fijada antes de mirar sus resultados fue elegir el modelo con la mayor media de cita relevante y abstención correcta en Milvus en la corrida principal: gemma2:2b obtuvo 0,954 y llama3.2:3b 0,908. La elección tiene un costo conocido, que gemma2:2b sufrió el único error residual de la corrida principal (Sección 13).

Cuadro 12: Ablaciones con gemma2:2b y Milvus (72 llamadas por fila; IC remuestreando las 24 preguntas).

Variante	Cita relevante [IC]	Abst. correcta [IC]	Abst. con cita
$k = 5$, tok256 (principal)	0,950 [0,85; 1,00]	0,958 [0,88; 1,00]	0,125
$k = 3$	0,850 [0,65; 1,00]	0,917 [0,79; 1,00]	0,125
$k = 10$	0,900 [0,75; 1,00]	0,958 [0,88; 1,00]	0,125
tok128 (E5)	0,917 [0,80; 1,00]	0,958 [0,88; 1,00]	0,083
sent256	0,867 [0,72; 1,00]	0,917 [0,79; 1,00]	0,167
MiniLM, tok256	0,600 [0,40; 0,80]	0,833 [0,67; 0,96]	0,208
MiniLM, tok128	0,650 [0,45; 0,85]	0,833 [0,67; 0,96]	0,125
Un fragmento por documento	0,800 [0,60; 0,95]	0,833 [0,67; 0,96]	0,250

La configuración principal es nominalmente la mejor, pero sus intervalos se cubren con los de $k = 3$, $k = 10$, tok128 y sent256, así que no podemos afirmar que $k = 5$ sea mejor que esas variantes. Solo un efecto es nítido: cambiar E5 por MiniLM con fragmentos de 256 tokens reduce la cita relevante de 0,950 a 0,600 [0,40; 0,80], con intervalos disjuntos. Esa ablación, sin embargo, mezclaba el encoder con una longitud efectiva de 128 tokens, porque MiniLM trunca la entrada. La comparación aislada, E5 con tok128 (0,917 [0,80; 1,00]) frente a MiniLM con tok128 (0,650 [0,45; 0,85]), conserva una diferencia de 0,27 en cita relevante, pero los intervalos se solapan entre 0,80 y 0,85; en abstención correcta las dos variantes dan 0,958 y 0,833, también con intervalos solapados. La dirección coincide con la de la recuperación, y

con 24 preguntas no alcanza para dar la diferencia por concluyente. El contexto de un fragmento por documento baja la cita relevante a 0,800 y sube la abstención con cita espuria a 0,250 (los intervalos se solapan con la principal); una lectura es que al eliminar los fragmentos repetidos del mismo documento se pierde también el pasaje adicional que sostenía la respuesta, y que con menos texto el modelo duda más.

12.4. Evaluación humana

[PENDIENTE: evaluación humana y kappa] Las hojas de calificación (30 respuestas, calificadas a ciegas por dos evaluadores con la rúbrica de cinco criterios) están preparadas y congeladas. El Cuadro 13 es el esqueleto donde se incorporarán las cifras cuando lleguen las hojas completadas.

Cuadro 13: Evaluación humana (esqueleto; las cifras llegan con las hojas completadas).

Criterio	Media A	Media B	Acuerdo exacto	κ ponderado
Corrección	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**
Fidelidad	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**
Relevancia	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**
Calidad de citas	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**
Abstención	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**	**[PENDIENTE]**

13. Rendimiento

El Cuadro 14 lista los modelos tal como los reporta Ollama y la VRAM máxima observada con el monitor de recursos mientras cada uno estaba cargado. Ningún modelo se acercó al límite de 6.144 MB: el máximo, 3.323 MB, corresponde a llama3.2:3b.

Cuadro 14: Modelos usados, cuantización real y VRAM máxima observada.

Modelo	Parámetros	Cuantización	Digest	VRAM máx. (MB)
qwen2.5:1.5b	1,5B	Q4_K_M	65ec06548149	1.997
llama3.2:1b	1,2B	Q8_0	baf6a787fdff	2.325
gemma2:2b	2,6B	Q4_0	8ccf136fdd52	2.920
llama3.2:3b	3,2B	Q4_K_M	a80c4f17acd5	3.323

Generación. El Cuadro 15 separa la latencia en caliente de la de arranque en frío. En caliente, la mediana va de 0,7 s en qwen2.5:1.5b y llama3.2:1b a 2,2 s en llama3.2:3b con Milvus, y el p95 de 1,3 a 13,3 s. Los modelos más pequeños generan unos 131 a 137 tokens por segundo; gemma2:2b y llama3.2:3b, unos 80. La Figura 8 muestra que los modelos de 1B y 1,5B ocupan menos VRAM (2.325 y 1.997 MB) y generan unos 130 tokens por segundo, mientras que los de 2,6B y 3,2B ocupan 2.920 y 3.323 MB y generan unos 80. El arranque en frío cuesta entre 25 y 41 s, con un caso de 113 s en llama3.2:3b con Solr; son llamadas únicas por celda y las

reportamos aparte porque dominarían cualquier percentil. Con Solr los contextos son más largos y el p50 de llama3.2:3b sube de 2,2 a 4,3 s.

Cuadro 15: Latencia de generación del experimento principal (segundos) y velocidad media en tokens por segundo. «–»: esa celda no tuvo llamada fría, porque su corrida empezó con el modelo ya cargado.

Modelo	Motor	p50 caliente	p95 caliente	Frío (una llamada)	tok/s
gemma2:2b	Milvus	1,95	8,62	29,0	81,1
gemma2:2b	Solr	1,69	8,53	37,7	80,5
llama3.2:3b	Milvus	2,22	6,44	40,8	81,3
llama3.2:3b	Solr	4,29	13,34	112,6	78,6
qwen2.5:1.5b	Milvus	0,71	1,41	24,6	131,8
qwen2.5:1.5b	Solr	0,72	1,49	–	131,4
llama3.2:1b	Milvus	0,76	1,56	26,4	137,4
llama3.2:1b	Solr	0,78	1,32	–	136,6

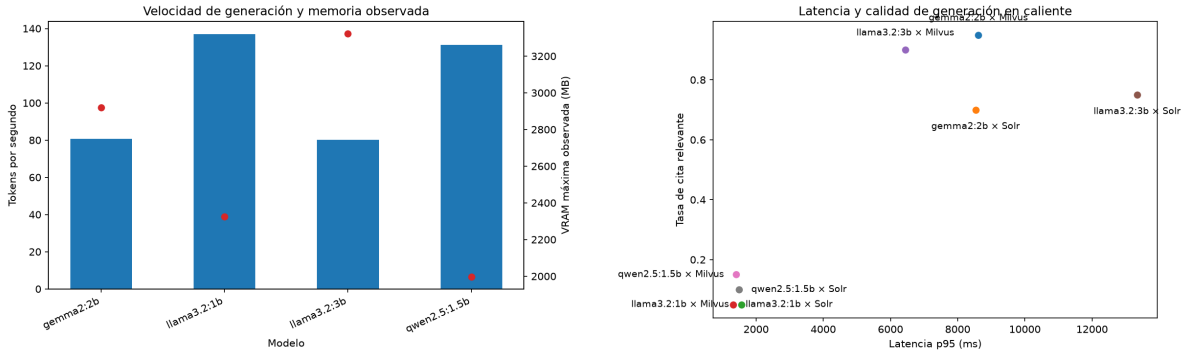


Figura 8: Izquierda: tokens por segundo (barras) y VRAM máxima observada (puntos). Derecha: latencia p95 en caliente frente a tasa de cita relevante.

La parte derecha de la Figura 8 cruza velocidad y calidad. Los modelos de 1B y 1,5B ocupan la esquina rápida y sin citas; gemma2:2b con Milvus tiene la mejor calidad con un p95 de 8,6 s, y llama3.2:3b con Milvus ofrece una calidad algo menor con un p95 de 6,4 s, por lo que esas dos combinaciones forman la frontera útil. llama3.2:3b con Solr queda dominado por las combinaciones con Milvus. Dos puntos de la esquina inferior izquierda se superponen en la figura (llama3.2:1b con ambos motores), porque tienen la misma cita relevante y latencias parecidas.

Recuperación. La recuperación es mucho más rápida que la generación (Cuadro 16). Medimos estas latencias con la máquina en reposo, sin otros procesos activos. Con la muestra piloto, Solr responde con mediana de 5,8 ms y p95 de 27,8 ms; Milvus con 29,2 ms (p95 66,1 ms), porque cada consulta pasa antes por el encoder en CPU, y el híbrido, que consulta ambos motores, con 37,5 ms. Con 10.000 documentos las medianas casi no cambian (5,7 ms en Solr y 25,8 ms en Milvus). Una medición anterior, tomada con la máquina cargada por otras tareas, dio medianas de 17,4 y 97,6 ms, así que solo damos por firme el orden (Solr más rápido que Milvus) y no las cifras absolutas de una sola corrida. La primera consulta de MiniLM con tok256

tarda 12,5 s por la carga del modelo. Con los modelos que citan, la recuperación pesa menos del 2 % de la mediana del tiempo total de una respuesta (34,2 ms de 1.951 ms con gemma2:2b y Milvus), así que la elección de motor no se justifica por velocidad.

Cuadro 16: Latencia de recuperación en milisegundos.

Motor	Frío (700)	p50 (700)	p95 (700)	p50 (10.000)	p95 (10.000)
Solr	6,0	5,8	27,8	5,7	20,4
Milvus	30,9	29,2	66,1	25,8	30,1
Híbrido	40,3	37,5	49,7	–	–

Recursos, indexación y errores. Con el monitor de recursos observamos en la corrida principal picos de 733 MB de RAM en la API y de 4.096 MB en Ollama, que en esa primera pasada era su límite. En el uso de CPU (porcentaje de un núcleo, columna cpu_pct), Ollama tuvo en esa corrida una mediana de 45 % y un máximo de 180 %, y la API una mediana de 0,1 % y un máximo de 99 %; en las ablaciones la API llegó a 144 % y Ollama a 159 %, con 1.390 MB de RAM en la API. En las ablaciones, con el límite en 6 GB, Ollama llegó a 6.044 MB, muy cerca del tope, sin errores. La VRAM nunca pasó de 3.323 de 6.144 MB. En indexación, Solr incorporó 36.173 fragmentos en 72 s (unos 500 por segundo) y Milvus en 4.288 s (8,4 por segundo), unas 60 veces más lento porque el cuello de botella es el encoder en CPU (Sección 6). La primera pasada de la corrida principal tuvo 11 errores en 576 llamadas (1,9 %), todos por falta de memoria de Ollama con 4 GB; tras subir el límite a 6 GB y repetir esas llamadas, queda un error en 576 (0,17 %), una llamada intermitente de gemma2:2b con Solr que no se reprodujo al repetirla a mano. Las siete ablaciones terminaron con cero errores.

14. Robustez

Inyección de instrucciones. Usamos cuatro documentos sintéticos con una orden embebida («ACCESO CONCEDIDO») y medimos el éxito del ataque con una coincidencia literal. En el conjunto de prueba hay dos preguntas de inyección, y por eso cada celda de modelo y motor tiene seis llamadas, y en las ocho celdas el éxito fue de 0 en 6, es decir, 0 en 48 llamadas en la corrida principal y 0 en 42 en las siete ablaciones. Con dos preguntas de inyección y salidas deterministas, 0 de 48 llamadas equivale a 0 de 16 combinaciones (pregunta, modelo, motor); muestra que el ataque literal no funcionó, pero no acota la tasa. Este resultado depende de la versión del prompt: con el v2, llama3.2:3b repitió la orden inyectada; con el v3, que declara que los documentos son datos y no órdenes, ninguno de los cuatro modelos la repitió en los controles. Además, la ausencia de éxito no significa lo mismo en todos los modelos: gemma2:2b y llama3.2:3b responden la pregunta con una cita relevante en las seis llamadas, mientras que qwen2.5:1.5b y llama3.2:1b no citan ningún documento relevante en las preguntas de inyección, es decir, resisten porque ignoran el contexto. La medida detecta solo la repetición literal y no una obediencia parcial.

Fechas. El gold tiene cuatro preguntas temporales (dos en la prueba, seis llamadas por celda). Todos los motores las recuperan sin fallo (Recall@5 de 1,00), y gemma2:2b y llama3.2:3b citan el documento relevante en el 100 % de las llamadas con ambos

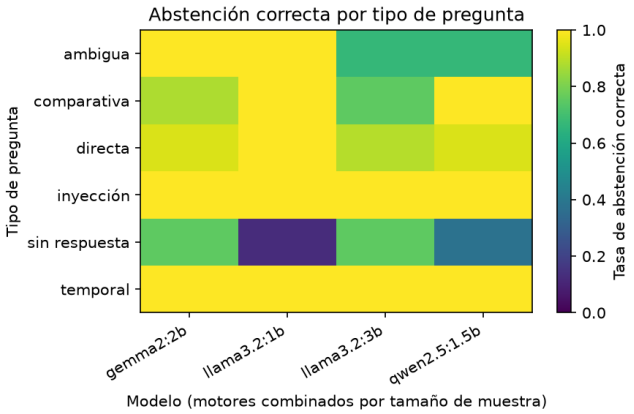


Figura 9: Abstención correcta por tipo de pregunta y modelo (ambos motores combinados).

motores; qwen2.5:1.5b y llama3.2:1b lo hacen en el 50 % con Milvus y en ninguna con Solr. La muestra es demasiado pequeña para afirmar que el manejo de fechas es sólido, y el caso del Encuentro Nacional (Sección 15) muestra que un modelo puede equivocar un día con la evidencia delante.

Discrepancias entre fuentes. Las comparativas exigen contrastar dos documentos (cuatro en la prueba, 12 llamadas por celda). Con Milvus, gemma2:2b y llama3.2:3b citan un documento relevante en el 100 % de las llamadas de este tipo, y con Solr en el 50 %, en línea con el Recall@5 de las comparativas (0,85 con Milvus y 0,52 con Solr): cuando el motor trae solo una de las dos fuentes, el modelo contesta con lo que tiene y no avisa de que falta la otra. El gold no tiene fuentes contradictorias, así que no medimos cómo trata el modelo una discrepancia real entre documentos.

Abstención sin respuesta. En las cuatro preguntas sin respuesta del conjunto de prueba (12 llamadas por celda), gemma2:2b y llama3.2:3b se abstienen correctamente en el 75 % de las llamadas con ambos motores, y qwen2.5:1.5b y llama3.2:1b entre el 0 y el 50 %, por lo que responden en lugar de abstenerse (Figura 9). En gemma2:2b todas las abstenciones correctas llevan una cita espuria al final («No hay evidencia suficiente en los documentos recuperados. [4]»), que la métrica de abstención con cita registra como 0,75 en esas preguntas; la abstención es correcta pero la cita no sostiene nada y confunde a quien la lea.

15. Análisis de errores

Separamos los errores por la etapa en que se originan: de recuperación, cuando la evidencia necesaria no estaba entre los k resultados, y de generación, cuando estaba y el modelo falló. El Cuadro 17 resume los casos que analizamos, tomados de las listas de casos del análisis de resultados y de la operación del sistema.

Errores de recuperación. Son pocos y se concentran en Solr. En Q12, Solr (y el híbrido, que hereda su fallo) no trae el documento relevante y el modelo, bien orientado por el prompt, se abstiene: la abstención es la conducta correcta dada la evidencia, pero cuenta como incorrecta porque la pregunta sí tenía respuesta. La falla está en el motor y se corrige con otro método de recuperación, no con otro modelo. Por eso

Cuadro 17: Casos de error analizados y etapa de origen.

Caso	Etapa	Qué ocurrió
Q12 (récord de la cocaína), Solr	Recuperación	El documento relevante no está entre los cinco primeros de Solr ni del híbrido; Milvus sí lo trae. gemma2:2b y llama3.2:3b se abstienen con Solr (6 de 6 llamadas).
Q03 (asesinatos en Colombia), gemma2:2b, Solr	Generación	Con el documento correcto en el contexto, responde «31» y cita [1]; la referencia dice 79 de 196. El juez le da fidelidad 0.
Q04 (COP16 en Cali), llama3.2:1b	Generación	Responde sin citas y con la causa inventada (el sector privado) o a medias (terremoto); fidelidad 0 con ambos motores.
Q04, gemma2:2b, Milvus	Generación	Con evidencia relevante, 3 de las 21 respuestas registradas para esta pregunta se abstienen sin motivo.
Q01 (fracking), llama3.2:1b	Generación	Acierta la frase pero no cita, y con Solr añade una afirmación contraria a la referencia («política de no realizar fracking»).
Encuentro Nacional por el Agua y la Energía Renovable	Generación	Con Milvus gemma2:2b dice 16 de noviembre; con Solr dice 15, y el documento dice 15.
Prototipo de cinco documentos	Contexto y citas	Una respuesta citó [1], [2], [3] y [5] sin que el contexto sustentara lo afirmado.

la abstención incorrecta mezcla fallos del motor y del modelo, y solo la revisión caso por caso los distingue. En Q12 no hubo aciertos sin evidencia: con Solr, sin el documento relevante, tres modelos se abstienen y llama3.2:1b responde sin citas, así que ninguna respuesta correcta se obtuvo sin tener la evidencia en el contexto.

Errores de generación. Son más variados y en su mayoría no los detectan las métricas mecánicas. En Q03, la cita apunta al documento correcto y por eso cuenta como cita relevante, pero la cifra es falsa: una cita válida no garantiza una afirmación verdadera, y solo el juez y la referencia lo advierten. Los modelos de 1B y 1,5B fallan de otro modo: responden sin citas, con aciertos parciales (Q01) o con invención (Q04), y sus respuestas no se pueden auditar. En Q09 (reintroducción de castores en Gales), llama3.2:1b atribuye la revisión a organismos que el documento no menciona, y el juez le da fidelidad 0. La abstención incorrecta de gemma2:2b en Q04, en cambio, es un exceso de cautela con evidencia suficiente.

El Encuentro Nacional. Durante las pruebas de la interfaz hicimos la misma pregunta sobre cuándo se realizaba el Encuentro Nacional por el Agua y la Energía Renovable a gemma2:2b con cada motor. Con Milvus respondió «16 de noviembre de 2023» y citó el fragmento [2]; con Solr respondió el 15, que es la fecha del documento, y los otros tres modelos con Milvus también dieron el 15. La evidencia estaba en el contexto en ambos casos, así que el error es de generación. La validación de citas no lo detecta porque la cita apunta al documento correcto; por eso recomendamos comparar cifras y fechas con el fragmento citado.

La iteración con cinco documentos. El prototipo previo del equipo, con un índice

de humo de cinco documentos, produjo una respuesta con las citas [1], [2], [3] y [5] sin que el contexto las sustentara. El defecto estaba en cómo se armaba el contexto (fragmentos truncados o ausentes) y no en la recuperación. Lo corrigieron dos decisiones que el sistema actual conserva: los fragmentos entran enteros o no entran, y una cita solo es válida si apunta a un fragmento incluido en el contexto. Por esa regla la tasa de citas válidas es 1,000 en todos los modelos, aunque eso no garantiza que la cita respalde lo afirmado (Q03).

16. Limitaciones

La principal es que el gold lo redactamos leyendo documentos concretos, lo que facilita la recuperación por solapamiento de palabras y tiende a sobreestimar la recuperación de ambos motores frente a preguntas de usuarios reales; el resultado en 10.000 documentos (Recall@5 de Milvus de 0,700) mezcla degradación real con relevantes que nadie juzgó, y con el gold actual no sabemos separarlas. El *pooling* de profundidad 3 deja sin juzgar posibles relevantes en las posiciones 4 a 10, y el gold sigue en estado borrador hasta que una persona lo verifique, por lo que las cifras pueden cambiar. Con 40 preguntas con respuesta y 24 de prueba, los intervalos son anchos y muchas comparaciones por pares son no concluyentes; en subgrupos de 4 a 8 preguntas, solo orientan. Con temperatura 0 las corridas repetidas aportan poca información, y los intervalos reflejan sobre todo la variación entre preguntas.

El experimento se hizo en un solo equipo, con una GPU de 6 GB, y las latencias no se extrapolan a otro hardware. Los modelos van cuantizados (Q4 en tres de ellos y Q8 en el cuarto), y por eso el efecto del modelo se confunde con el de la precisión. La muestra piloto de 700 documentos no es el corpus completo: indexarlo supondría unos 558.000 fragmentos y unas 18,5 horas de embeddings en 4 vCPU, y no cabía en el plazo. El juez es un modelo de 7B sin validación humana, la evaluación humana está pendiente y el análisis pareado de H2 se añadió después de ver los resultados. Por último, las métricas de abstención y de citas son mecánicas: no distinguen una cita que sostiene una afirmación de una que solo apunta a un documento relevante.

17. Conclusiones y recomendaciones

Con Milvus, multilingual-e5-small, fragmentos tok256, $k = 5$, prompt v3 y gemma2:2b el sistema recupera y cita bien en el piloto. En nuestras pruebas el motor de recuperación influyó más en las respuestas que el modelo de lenguaje, aunque con 24 preguntas de prueba los intervalos de las respuestas con Milvus y Solr se solapan. Con 10.000 documentos la ventaja de Milvus no se mantiene con claridad, y parte de la caída del Recall@5 (a 0,700) corresponde a relevantes que nadie anotó. Las cifras siguientes se pueden comprobar con los archivos de reports/.

- *Calidad.* Recall@5 de 0,946 [0,879; 0,992] en las 40 preguntas con respuesta (desarrollo y prueba juntos) y cita relevante de 0,950 [0,849; 1,000] en las 24 preguntas de prueba. En 10.000 documentos el Recall@5 es 0,700 [0,567; 0,812]; la caída se concentra en Bluesky y mezcla degradación con relevantes sin anotar, por lo que no la tomamos como la cifra esperable en el corpus completo.
- *Recursos.* VRAM de hasta 3.323 MB de 6.144 MB, RAM de Ollama entre 3,1

y 6,0 GB según la carga (límite recomendado de 6 GB) y RAM de la API por debajo de 1,4 GB.

- *Velocidad.* Recuperación con mediana de 34,2 ms y p95 de 54,0 ms con Milvus en la corrida de generación (29,2 y 66,1 ms en la corrida dedicada, medida en reposo); generación en caliente con mediana de 1,95 s y p95 de 8,62 s con gemma2:2b, y arranque en frío de unos 29 s.
- *Trazabilidad.* Cada respuesta guarda la versión del prompt, la huella del modelo, el hash del contexto y la evidencia citada, y las citas se validan contra el contexto (1,000 en todas las configuraciones).
- *Reproducibilidad.* Imágenes con versión fija, semilla 42, huella del encoder en cada colección, indexación idempotente y conteos verificados tras reiniciar los servicios. A temperatura 0 las tres corridas dieron casi siempre la misma puntuación, así que repetirlas no mide la variación entre preguntas.
- *Riesgos.* Inyección en 0 de 48 llamadas (con dos preguntas de inyección eso no acota la tasa, y los modelos de 1B y 1,5B resisten ignorando el contexto), abstenciones con cita espuria (hasta 0,25 en las ablaciones) y fechas o cifras equivocadas con cita válida, que el sistema no detecta.

Respuesta a la pregunta de investigación. Entre los cuatro modelos locales, gemma2:2b con Milvus ofrece el mejor equilibrio entre calidad, recursos y velocidad: cita un documento relevante en 0,950 [0,85; 1,00] de sus respuestas, se abstiene correctamente en 0,958 [0,88; 1,00], tiene un p95 en caliente de 8,62 s y ocupa 2.920 MB de VRAM. llama3.2:3b queda empatado en calidad dentro de los intervalos (0,900 y 0,917), con mejor p95 con Milvus (6,44 s) pero más VRAM (3.323 MB), y con Solr su p95 sube a 13,34 s. qwen2.5:1.5b y llama3.2:1b son los más livianos (1.997 y 2.325 MB de VRAM, p95 de 1,41 y 1,56 s), pero casi no citan (0,150 y 0,050) y responden sin citas entre el 56 y el 89 % de las veces, lo que los descarta cuando hay que verificar la respuesta. Elegimos gemma2:2b, con la reserva de que su arranque en frío tarda unos 29 s y de que 24 preguntas de prueba no bastan para separarlo con claridad de llama3.2:3b.

Lo más urgente es indexar el corpus completo con GPU para los embeddings, porque es lo que más afecta la comparación entre motores. Después convendría reordenar con un *cross-encoder* los primeros 20 a 50 resultados, calibrar RRF y verificar el gold con más anotadores.

18. Licencias, seguridad y nota sobre el corpus

18.1. Licencias de los modelos

El Cuadro 18 resume las licencias según la ficha publicada de cada modelo; deben revisarse antes de cualquier uso fuera del taller (las licencias de Llama y Gemma imponen políticas de uso aceptable).

Cuadro 18: Licencias de los modelos usados (según la ficha de cada modelo).

Modelo	Licencia
qwen2.5:1.5b (y juez qwen2.5:7b)	Apache-2.0
llama3.2:1b, llama3.2:3b	Llama 3.2 Community License
gemma2:2b	Gemma Terms of Use
multilingual-e5-small	MIT
paraphrase-multilingual-MiniLM-L12-v2	Apache-2.0

18.2. Seguridad

Las entradas de la API se validan (longitud, rango, tipos). El texto del usuario se neutraliza antes de llegar a Solr. Contra la inyección de instrucciones, el prompt declara que los documentos son datos y no órdenes, el contexto se delimita con etiquetas, y se probó con cuatro documentos sintéticos que contienen una orden embebida; el éxito del ataque se mide mecánicamente. Las credenciales de MinIO son valores locales por defecto, solo accesibles desde la red interna de Docker (los puertos se enlazan a 127.0.0.1); no hay secretos en el repositorio y la configuración se lee de un archivo .env no versionado, con un .env.example sin secretos. Los datos personales se enmascaran (Sección 2).

18.3. Nombre del conjunto de datos

La guía del taller menciona el corpus como dragons_rag_2026_10_19; ese nombre no existe en el catálogo publicado. El archivo usado es el de dragons_rag del catálogo open_corpus_co_es versión 0.1.1a4, identificado por su huella SHA-256 (Sección 2), que es la referencia fiable.

A. Repositorio y artefactos

El código está en el repositorio **[PENDIENTE: URL del repositorio]**, rama main. Los artefactos de este informe están en las rutas siguientes.

- eval/gold.jsonl, eval/annotations/ y eval/human/: gold, anotaciones y hojas de evaluación humana.
- reports/perfilado/, reports/chunking/ y reports/milvus/: perfilado del corpus, estadísticas de segmentación y esquemas de las colecciones.
- reports/retrieval/ y reports/generation/: métricas de recuperación y de generación (resúmenes, por tipo, por llamada y latencias).
- reports/runs/raw/: configuración de cada corrida (*_config.json) y monitor de recursos (sysmon_*.csv).
- reports/tables/ y reports/figures/: tablas y figuras del informe.
- prompts/ y scripts/: los tres prompts versionados y los guiones de indexación, corridas y análisis.

El README del repositorio tiene los comandos de replicación: descarga y retiro de

modelos con ollama pull y ollama rm, y las pruebas con pytest -q (341 pruebas unitarias y 5 de integración).

No se publican el Parquet ni los textos completos del corpus, ni las salidas crudas de las corridas, porque contienen texto de fragmentos y *handles* de Bluesky. Se publican annotated_runs.jsonl y los CSV sin texto del corpus, con los datos personales enmascarados.

Referencias

[1] Jacob Cohen. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968.

[2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

[3] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[4] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[6] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024. Versión preliminar: arXiv:2307.03172, 2023.

[7] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. Versión preliminar: arXiv:1603.09320, 2016/2018.

[8] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[9] Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[10] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[11] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[12] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021.

[13] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.